

Task 3 Write-Up: Adversity Prediction Model

1. Feature Vector Ordering

To ensure the prediction engine strictly matches the training architecture, the input feature vector X must be passed in the exact following indexed order:

- [0] Volume (Float/Int): The size of the trade.
- [1] Spread (Float): The bid-ask spread at the time of the trade.
- [2] Hour (Int): The extracted hour of the trade execution.
- [3] Minute (Int): The extracted minute of the trade execution.
- [4] Trade_Aggressiveness (Float): The premium paid to cross the spread.
- [5] Side (Int): +1 for Buy, -1 for Sell.
- [6] Name_TE (Float): The Target Encoded historical adversity probability of the client.

2. Feature Selection and Justification

In financial micro-structure data, avoiding "Data Leakage" is the primary challenge. We strictly isolated future mid-prices (M_5 to M_{30}) for target creation only, ensuring all features are strictly known at time $t = 0$.

- **Trade Aggressiveness:** Calculated as $Side \times (TradePrice - M_0)$. This is a crucial market-making feature. If a client is willing to pay a massive premium over the mid-price to execute immediately, it strongly implies they possess directional information (high toxicity).
- **Name_TE (Target Encoding):** Client identity is the strongest predictor of adversity (e.g., Institutional Client F vs. Retail Client A). However, using the whole dataset to calculate a client's risk profile causes future data leakage. To solve this, we calculated the client's historical adversity percentage *strictly on the 60% Train Set*. For live predictions, this map is applied forward, defaulting to the global market average for unseen clients.
- **Time Features (Hour, Minute):** Market toxicity changes across regimes (e.g., volatile market opens vs. quiet lunch hours). Extracting cyclic time variables allows the model to map these intraday liquidity shifts.

3. Choice of Model and Rationale

We chose the **HistGradientBoostingClassifier** (Histogram-based Gradient Boosting).

- **Rationale:** Financial tick data is highly non-linear and incredibly noisy. Linear models (like Logistic Regression) fail to capture complex feature interactions, while Deep Learning models are too slow for millisecond-level execution and prone to overfitting noisy data. Gradient Boosting provides the optimal balance: it natively handles non-linear relationships, is robust to outliers, and executes fast enough for a live trading environment.
- **Chronological Splitting Constraint:** A standard random split (e.g., `train_test_split`) would allow the model to use May's data to predict January's data, invalidating the model. We mandated a strict chronological sort by `datetime` before applying the 60/20/20 (Train/Validation/Test) split, proving the model possesses genuine predictive power on out-of-sample future data.

4. Results and Financial Justification

The model’s performance was averaged across all six horizons ($\tau \in \{5, 10, 15, 20, 25, 30\}$) on the strict chronological test set:

Split	Accuracy	Precision	Recall	Log Loss
Train	56.55%	60.45%	24.14%	0.680
Validation	55.63%	53.34%	23.24%	0.684
Test	55.73%	51.88%	23.11%	0.684

In traditional data science, 55% accuracy might seem low, but in quantitative finance, it represents a massive statistical edge.

- **Log Loss & Accuracy:** A completely random guess yields 50% accuracy and a log loss of 0.693. Achieving 55.7% and a log loss of 0.684 out-of-sample proves the model has a highly calibrated, profitable edge over random market movement.
- **Precision over Recall:** In externalization, precision is our most vital metric. If precision is low, we accidentally hedge safe retail flow, needlessly giving away our spread revenue. A test precision of $\sim 51.9\%$ means that when our system fires the hedge protocol, it is genuinely intercepting a toxic trade over half the time.
- **Recall Constraint:** Catching $\sim 23.1\%$ of all adverse trades is an excellent optimization. Attempting to force higher recall would make the model overly paranoid, destroying profitability by hedging everything. Catching the top quartile of the most obvious toxic flow protects our downside inventory risk while maximizing captured spread.